

分布式任务调度平台 XXL-JOB 开发示例与核心原理

 小翎哥 收录于 [Posts](#)

□ 2023-12-06 □ 2024-03-12 □ 约 5900 字 □ 预计阅读 12 分钟

公告：本域名将于 2025 年 5 月 31 日到期，本站所有内容将逐步迁移到 <https://iling.me>。

被迁移的链接将保持一年的重定向，如果您收藏了本站的内容，请保存对应的新链接，以免将来无法访问。

XXL-JOB 是由许雪里大佬开源的一款分布式任务调度平台。其凭借着简单易操作的特性，赢得了许多小型自研公司的青睐，甚至像阿里这样的大型企业也将 XXL-JOB 集成到了自己的 ScheduleX 中。然而，这么优秀的一款开源软件，我竟然一直没有了解过。因此，趁着最近有空，我决定深入学习一下 XXL-JOB 的核心内容，遂著此文。

1 基础组件

XXL-JOB 主要包括**调度中心**和**执行器**两大基础组件。

| 1.1 调度中心

调度中心是一个单独的 Web 服务，位于源码中的 `xxl-job-admin` 模块。它主要用于触发定时任务，而且还提供了一个简易的后台管理页面，方便用户去管理定时任务。

调度中心依赖数据库，且支持横向扩展。然而，每个调度中心的节点必须连接到同一数据库，因此，即使数据库启用了主从复制，节点仍然需要指向主库。

调度中心的节点是**无状态**的，且它们之间不进行任何通信，数据仅通过唯一的数据库进行同步。

| 1.2 执行器

执行器是一个**嵌入式** HTTP 服务，位于源码中的 `xxl-job-core` 模块，通常由第三方工程引用该模块，并在其内部配置执行器。该执行器通过 Netty 框架监听来自调度中心的命令，并执行相应的操作。

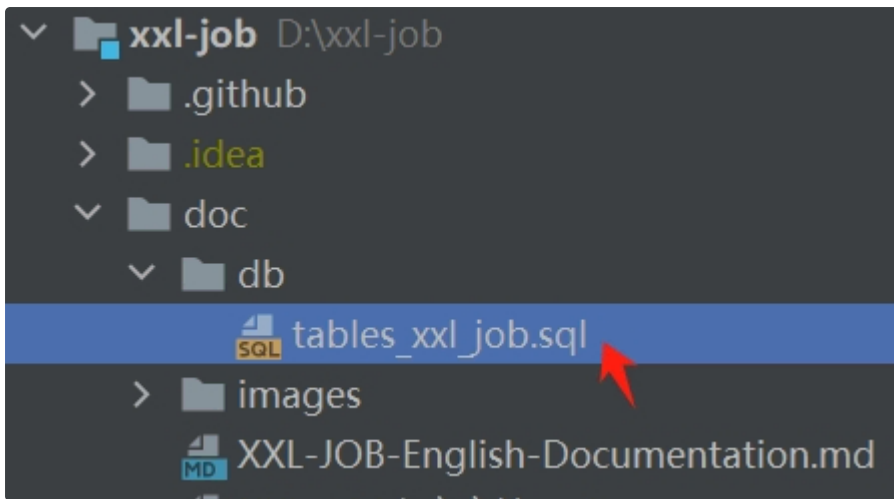
2 开发示例

| 2.1 搭建调度中心

首先，下载 XXL-JOB 的源码：

<https://github.com/xuxueli/xxl-job.git>

然后，部署一个 MySQL Server，并执行源码工程里 `doc/db/tables_XXL_JOB.sql` 内的 SQL，完成数据库的初始化：



最后，修改 `src/main/resources/application.properties` 中的数据源配置，并启动调度中心：

plaintext

```
1 ### xxl-job, datasource
2 spring.datasource.url=jdbc:mysql://xxx.xxx.com:3306/xxl_job?useUnicode=true
3 # 修改用户名、密码
4 spring.datasource.username=username
5 spring.datasource.password=password
6 # mysql 驱动
7 spring.datasource.driver-class-name=com.mysql.jdbc.Driver
```

警告：如果 MySQL Server 部署在远端导致连接超时，还需要修改连接池的配置，增大超时时间。

调度中心启动后，就可以登录控制台了，默认的 URL 为 `http://localhost:8080/xxl-job-admin/toLogin`，默认的账号密码为 `admin` & `123456`：



2.2 创建执行器与任务

首先，在 SpringBoot 工程的 pom.xml 中引入 xxl-job-core 模块：

□ XML

```
1 <dependency>
2     <groupId>com.xuxueli</groupId>
3     <artifactId>xxl-job-core</artifactId>
4     <version>2.4.1-SNAPSHOT</version>
5 </dependency>
```

然后，在配置类中注册一个 `XxlJobSpringExecutor` 执行器：

□ Java

```
1 @Configuration
2 public class XxlJobConfiguration {
3     @Bean
4     public XxlJobSpringExecutor xxlJobExecutor() {
5         XxlJobSpringExecutor xxlJobSpringExecutor = new XxlJobSpringExecutor();
6         // 设置调用中心的连接地址
7         xxlJobSpringExecutor.setAdminAddresses("http://localhost:8080/xxl-job-admin");
8         // 设置执行器的名称
9         xxlJobSpringExecutor.setAppname("my-xxljob-executor-demo");
10        // 设置执行器的通信端口
11        xxlJobSpringExecutor.setPort(9999);
12        // 设置安全访问 token
13        xxlJobSpringExecutor.setAccessToken("default_token");
14        // 任务执行日志存放的目录
15        xxlJobSpringExecutor.setLogPath("/data/applogs/xxl-job/jobhandler");
16        return xxlJobSpringExecutor;
17    }
18 }
```

最后，创建一个 **Bean 模式** 的任务，并启动 `SpringBootApplication`：

□ Java

```
1 @Component
2 public class TestJob {
3     private static final Logger logger = LoggerFactory.getLogger(TestJob.class);
4
5     @XxlJob("TestJob")
6     public void testJob() {
7         logger.info("TestJob 任务成功");
8     }
9 }
```

- **@XxlJob** 注解：指定任务的 Bean，参数为任务的 BeanName，下文配置 `JobHandler` 时需要这个参数。

完成以上步骤后，接下来我们只需要在 XXL-JOB-ADMIN 控制台配置刚刚创建的执行器与任务，就可以执行任务了。

2.3 在控制台配置执行器与任务

首先，点击**执行器管理**，注册上述配置的执行器：

编辑执行器

AppName*

my-xxljob-executor-demo

名称*

测试执行器

注册方式*

☒ 自动注册 ☐ 手动录入

机器地址*

请输入执行器地址列表，多地址逗号分隔

保存

取消

- **AppName** 必须设置为 `xxlJobSpringExecutor.setAppname()` 中设置的英文名；
- **名称** 表示中文别名，可随意设置。

然后，点击**任务管理**，注册上述配置的 `TestJob` 任务，运行模式为 Bean：

新增

基础配置

执行器*

测试执行器

任务描述*

测试任务

负责人*

Ling

报警邮件*

请输入报警邮件，多个邮件地址则逗号分隔

调度配置

调度类型*

CRON

Cron*

*****?

任务配置

运行模式*

BEAN

JobHandler*

TestJob

任务参数*

请输入任务参数

高级配置

路由策略*

第一个

子任务ID*

请输入子任务的ID,如存在多个则逗号分隔

调度过期策略*

忽略

阻塞处理策略*

单机串行

任务超时时间*

任务超时时间，单位秒，大于零时生效

失败重试次数*

失败重试次数，大于零时生效

保存

取消

最后，点击操作按钮，启动定时任务，便可在调度日志一栏查看执行结果：

调度日志

执行器	全部	任务	全部	状态	全部	调度时间	2023-12-17 00:00:00 - 2023-12-17	搜索	清理
每页	10	条记录							
任务ID	调度时间	调度结果	调度备注	执行时间	执行结果	执行备注	操作		
2	2023-12-17 17:23:41	成功	查看	2023-12-17 17:23:45	成功	无	操作		
2	2023-12-17 17:23:36	成功	查看	2023-12-17 17:23:46	成功	无	操作		
2	2023-12-17 17:23:32	成功	查看	2023-12-17 17:23:35	成功	无	操作		

可见，XXL-JOB 成功运行了我们的定时任务。不过在上文中，我们设置的 CRON 表达式为每秒执行一次，而图中所示的每次执行的时间间隔并不准确，接下来我们详细分析一下 XXL-JOB 的核心原理。

3 执行器的核心原理

| 3.1 执行器的启动过程

执行器的核心是 `XxlJobExecutor`，XXL-JOB 默认提供了两个执行器实现：

`XxlJobSimpleExecutor` 和 `XxlJobSpringExecutor`。这里我们只学习最常用的

`XxlJobSpringExecutor`，也就是上文我们在 Demo 的配置类中注册的那个 Bean。

该类实现了 `SmartInitializingSingleton` 接口，所以在 Bean 实例化完成后会调用其 `afterSingletonsInstantiated()` 方法，初始化并启动执行器：

Java XxlJobSpringExecutor

```
1 @Override
2 public void afterSingletonsInstantiated() {
3     ...
4     try {
5         super.start();
6     } catch (Exception e) {
7         throw new RuntimeException(e);
8     }
9 }
```

该方法会执行父类 `XxlJobExecutor` 中的 `start()`：

Java XxlJobExecutor

```
1 public void start() throws Exception {
2     // 日志、回调线程初始化
3     ...
4     // 初始化内嵌执行器
5     initEmbedServer(address, ip, port, appname, accessToken);
6 }
```

`XxlJobExecutor.start()` 的核心是 `initEmbedServer()` 方法，该方法会利用 Netty 框架启动一个 HTTP Server，并将自己注册到调度中心。核心实现如下：

Java EmbedServer

```
1 public void start(final String address, final int port, final String appname) {
2     // [1] 创建执行器
3     executorBiz = new ExecutorBizImpl();
4     // [2] 通过子线程启动内嵌服务
5     thread = new Thread(new Runnable() {
6         @Override
7         public void run() {
8             // [4] 创建 Netty 的主从 Reactor 线程，以及用于执行任务的线程池
9             EventLoopGroup bossGroup = new NioEventLoopGroup();
10            EventLoopGroup workerGroup = new NioEventLoopGroup();
11            ThreadPoolExecutor bizThreadPool = new ThreadPoolExecutor(...);
12            try {
13                // [5] 启动内嵌服务
14                ServerBootstrap bootstrap = new ServerBootstrap();
15                bootstrap.group(bossGroup, workerGroup)
16                    .channel(NioServerSocketChannel.class)
17                    ...// 添加 childHandler EmbedHttpServerHandler
18                // [6] bind
19                ChannelFuture future = bootstrap.bind(port).sync();
20                // [7] 将当前执行器注册到调度中心 (向每个 admin 都发送 post 请求)
```

```

21         startRegistry(appname, address);
22         // [8] 阻塞直到服务结束
23         future.channel().closeFuture().sync();
24     } catch ...
25     }
26 });
27 // [3] 子线程以守护模式启动，开始执行内嵌服务启动流程
28 thread.setDaemon(true);
29 thread.start();
30 }

```

执行器启动后，便可接收来自调度中心的 HTTP 请求。当执行器收到来自调度中心的指令时，会把请求交给 `ExecutorBiz` 处理。`ExecutorBiz` 定义了执行器支持的所有方法：

□ Java

```

1 public interface ExecutorBiz {
2     // 发送心跳
3     public ReturnT<String> beat();
4     // 发送空闲心跳
5     public ReturnT<String> idleBeat(IdleBeatParam idleBeatParam);
6     // 启动一个任务
7     public ReturnT<String> run(TriggerParam triggerParam);
8     // 终止一个任务
9     public ReturnT<String> kill(KillParam killParam);
10    // 获取执行器 log
11    public ReturnT<LogResult> log(LogParam logParam);
12 }

```

`ExecutorBiz` 有两个实现类，分别是 `ExecutorBizImpl` 和 `ExecutorBizClient`。其中：

- `ExecutorBizImpl` 是由执行器使用，负责处理来自调度中心的请求；
- `ExecutorBizClient` 由调度中心使用，负责发送相应的命令。

3.2 任务的注册过程

定时任务的抽象类是 `IJobHandler`，XXL-JOB 提供了三个默认实现：

- `MethodJobHandler`：用于执行 `BEAN` 模式创建的任务；
- `GlueJobHandler`：用于执行 `GLUE` 模式下的 Java 任务；
- `ScriptJobHandler`：用于执行除 `GLUE (Java)` 模式外的其它 `GLUE` 脚本任务，例如 `GLUE (Shell)`、`GLUE (Python)` 等。

本文我们只介绍 `MethodJobHandler` 的注册流程。XXL-JOB 提供了一个 `@XxlJob` 注解，执行器启动时会从 Spring 容器中寻找被该注解标记的 Bean，调用链路如下：

□ Java

```

1 XxlJobSpringExecutor#afterSingletonsInstantiated
2 |

```

```

3  +-> // 扫描 Spring 容器中被 @XxlJob 标注的 Bean
4      XxlJobSpringExecutor#initJobHandlerMethodRepository
5      |
6  +-> // 将这些 Bean 注册到 Map
7      XxlJobExecutor#registJobHandler(
8          com.xxl.job.core.handler.annotation.XxlJob,
9          java.lang.Object,
10         java.lang.reflect.Method)
11     |
12     +-> XxlJobExecutor#registJobHandler(
13         java.lang.String,
14         com.xxl.job.core.handler.IJobHandler)

```

`XxlJobExecutor` 内部维护了一个 map 容器 `jobHandlerRepository` ,
`initJobHandlerMethodRepository()` 中扫描到的 Bean 最终会由 `registJobHandler()`
封装成一个 `MethodJobHandler` 对象, 并注册到该容器中 :

□ Java

`XxlJobExecutor`

```

1  // 定时任务容器
2  private static ConcurrentMap<String, IJobHandler> jobHandlerRepository
3      = new ConcurrentHashMap<String, IJobHandler>();
4
5  // 注册定时任务到容器
6  public static IJobHandler registJobHandler(String name, IJobHandler jobH
7      ...
8      return jobHandlerRepository.put(name, jobHandler);
9  }
10 protected void registJobHandler(XxlJob xxlJob, Object bean, Method execu
11     ...
12     // registry jobhandler
13     registJobHandler(name,
14         new MethodJobHandler(bean, executeMethod, initMethod, destroyMet
15 }

```

3.3 任务的执行过程

上文我们讲过, 执行器启动内嵌服务时, 向 Channel 注册了一个

`EmbedHttpServerHandler` , 它是 `io.netty.channel.SimpleChannelInboundHandler`
的实现类, 当有数据可读时会调用其 `channelRead0` 方法。

`EmbedHttpServerHandler#channelRead0()` 会解析来自调度中心的请求, 并交给线程池
处理 :

□ Java

`EmbedHttpServerHandler`

```

1  @Override
2  protected void channelRead0(final ChannelHandlerContext ctx, FullHttpReq
3      // [1] 解析调度中心的指令
4      String requestData = msg.content().toString(CharsetUtil.UTF_8);
5      String uri = msg.uri(); // restfull, url 就是指令类型, 例如 run、beat...

```



```

6      HttpMethod httpMethod = msg.method();
7      boolean keepAlive = HttpUtil.isKeepAlive(msg);
8      String accessTokenReq = msg.headers().get(XxlJobRemotingUtil.XXL_JOB
9      // [2] 交给线程池处理命令
10     bizThreadPool.execute(new Runnable() {
11         @Override
12         public void run() {
13             // [3] 执行命令
14             Object responseObj = process(httpMethod, uri, requestData, a
15             // [4] 返回响应
16             String responseJson = GsonTool.toJson(responseObj);
17             writeResponse(ctx, keepAlive, responseJson);
18         }
19     });
20 }

```

执行命令的核心是 `process()` 方法：

```

❑ Java      EmbedHttpServerHandler

1  private Object process(HttpMethod httpMethod, String uri, String request
2      // 参数校验
3      ...
4      // services mapping
5      try {
6          switch (uri) {
7              case "/beat":
8                  ...
9              case "/idleBeat":
10                 ...
11             case "/run":
12                 TriggerParam triggerParam = GsonTool.fromJson(requestData
13                 return executorBiz.run(triggerParam);
14             case "/kill":
15                 ...
16             case "/log":
17                 ...
18             default:
19                 ...
20         }
21     } catch ...
22 }

```

`process()` 通过请求的 url 来区分命令类型，进而调用 `ExecutorBiz` 实现类中的方法处理命令。以运行任务的 `run` 命令为例，该方法的实现如下：

```

❑ Java      ExecutorBizImpl

1  @Override
2  public ReturnT<String> run(TriggerParam triggerParam) {
3      // [1] 从缓存 jobThreadRepository 中取出 jobThread，第一次调用时为 null

```

```

4      JobThread jobThread = XxlJobExecutor.loadJobThread(triggerParam.getJobId());
5      // 从 jobThread 中取出 jobHandler
6      IJobHandler jobHandler = jobThread != null ? jobThread.getHandler() : null;
7      String removeOldReason = null;
8      // [2] 获取任务类型
9      GlueTypeEnum glueTypeEnum = GlueTypeEnum.match(triggerParam.getGlueType());
10     if (GlueTypeEnum.BEAN == glueTypeEnum) {
11         // [3] 根据 BEAN Name 取出上文注册在 jobHandlerRepository 中的 Handler
12         IJobHandler newJobHandler = XxlJobExecutor.loadJobHandler(triggerParam.getGlueType());
13         // [4] 如果 jobThreadRepository 缓存过期
14         if (jobThread != null && jobHandler != newJobHandler) {
15             ...
16             jobThread = null;
17             jobHandler = null;
18         }
19         // [5] 如果 jobHandler 为空，缓存新获取到的 jobHandler
20         if (jobHandler == null) {
21             jobHandler = newJobHandler;
22             if (jobHandler == null) {
23                 return ...
24             }
25         }
26     }
27     } else if ...
28
29     // [6] 执行拒绝策略
30     if (jobThread != null) {
31         ...
32     }
33     // [7] 创建一个封装了 jobHandler 的 JobThread，注册到缓存
34     if (jobThread == null) {
35         jobThread = XxlJobExecutor.registJobThread(triggerParam.getJobId(), jobHandler);
36     }
37     // [8] 将参数添加到 Trigger 队列，等待异步执行
38     ReturnT<String> pushResult = jobThread.pushTriggerQueue(triggerParam.getTriggerTime());
39     return pushResult;
40 }

```

- **[1]**：进入该方法时，首先尝试从缓存 `jobThreadRepository` 中获取 `jobThread`，如果 `jobHandler` 不为 null，再从中取出 `jobHandler`；
- **[2]**：根据任务类型处理对应的指令，这里我们重点介绍 BEAN 类型任务：
 - **[3]**：根据调度中心配置的 BEAN Name 获取上文注册到缓存 `jobHandlerRepository` 中的 `jobHandler`；
 - **[4]**：如果缓存 `jobThreadRepository` 中存在 `jobThread`，但其内部的 `jobHandler` 与 `jobHandlerRepository` 中的不一致，则清空 `jobThread` 和 `jobHandler` 的引用；

- `jobHandlerRepository` 中缓存了与 Bean Name 对应的 `JobHandler`，这个对应关系是永远不变的；
- `jobThreadRepository` 中缓存了与任务 ID 对应的 `JobThread`，而 `JobThread` 中又封装了 `JobHandler`，可以通过控制台动态修改任务 ID 对应的 `JobHandler`。

因此 `jobThreadRepository` 中缓存的 `JobThread` 有可能失效，所以这里需要更新缓存。

- **[5]**：如果 `jobHandler` 为 null，则将 `jobHandlerRepository` 中的 `newJobHandler` 赋值给 `jobHandler`；
- **[6]**：根据用户配置，执行拒绝策略；
- **[7]**：如果 `jobThread` 为 null，表明是第一次执行该任务，或者在第四步清空过 `jobThread`，所以这里创建一个新的 `jobThread` 并注册到 `jobThreadRepository`；
- **[8]**：将参数添加到阻塞队列 `triggerQueue`，然后直接返回。

由于定时任务的执行可能十分耗时，而执行器作为一个 HTTP Server，不适合长时间阻塞（否则会触发超时）。所以上述代码在第 8 步将参数添加到阻塞队列后就直接返回了，`JobThread` 在自己的循环中再不断拉取阻塞队列中的任务进行处理：

```

❑ Java JobThread
1 // com.xx1.job.core.thread.JobThread#run
2 ...
3 while(!toStop){
4     triggerParam = triggerQueue.poll(3L, TimeUnit.SECONDS);
5     ...
6     handler.execute(); // 执行任务
7     ...
8 }
9 ...

```

执行器收到触发指令后会直接返回 HTTP 响应，具体的任务是异步处理的，当任务执行完毕后，还需要发送回调请求将执行结果上报调度中心，这个过程也是异步的：

```

❑ Java JobThread
1 // com.xx1.job.core.thread.JobThread#run
2 ...
3 finally {
4     ...
5     // 回调，发送任务执行结果
6     TriggerCallbackThread.pushCallBack(new HandleCallbackParam(
7         triggerParam.getLogId(),
8         triggerParam.getLogDateTime(),
9         XxlJobContext.getXxlJobContext().getHandleCode(),
10        XxlJobContext.getXxlJobContext().getHandleMsg() )
11    );
12    ...

```

```
13 }
14 ...
```

了解了执行器的核心内容后，接下来我们分析 XXL-JOB 的重中之重：调度中心。

4 调度服务的核心原理

| 4.1 任务调度线程

调度中心会在 `XxlJobAdminConfig` 的后置处理器中启动一个调度线程，该线程会每秒查询一次 `xxl_job_info` 表内 `nowTime + PRE_READ_MS` 之前的待处理任务，最多 `preReadCount` 个：

□ Java

JobScheduleHelper

```
1 // 调度中心支持集群部署，所以每个节点在处理前要通过锁表抢占资源
2 preparedStatement = conn.prepareStatement(
3     "select * from xxl_job_lock where lock_name = 'schedule_lock' for update";
4 preparedStatement.execute();
5 // 查询 nowTime + PRE_READ_MS 之前的待触发任务，最多 preReadCount 个
6 List<XxlJobInfo> scheduleList = XxlJobAdminConfig
7     .getAdminConfig().getXxlJobInfoDao()
8     .scheduleJobQuery(nowTime + PRE_READ_MS, preReadCount);
```

- `xxl_job_lock`：调度中心支持集群部署，每个节点在处理前通过锁表抢占资源；
- `nowTime + PRE_READ_MS`：未来 5s 的时间窗口；
- `preReadCount`：每次最多查询任务数，计算方法： $\text{treadpool-size} * \text{trigger-qps}$ (each trigger cost 50ms, $\text{qps} = 1000/50 = 20$)。

拿到待处理的任务列表后，调度线程会将这些任务根据触发时间划分为三个部分：

1. 任务过期时间已经超过 5s；
2. 任务已经过时，但是过时时间不足 5s；
3. 任务未超时，且下次触发时间在未来 5s 内。

□ 注意

调度线程理论上一秒轮训一次，但实际的循环间隔并不精确：

□ Java

```
1 // Wait seconds, align second
2 if (cost < 1000) { // scan-overtime, not wait
3     try {
4         // pre-read period: success > scan each second; fail > skip t
5         TimeUnit.MILLISECONDS.sleep((preReadSuc?1000:PRE_READ_MS) - S
6     } catch (InterruptedException e) {
7         if (!scheduleThreadToStop) {
8             logger.error(e.getMessage(), e);
9         }
10    }
```

```
10     }  
11 }
```

原因有以下几个：

- 如果当次任务执行时间超过了 1s，那么下次轮训自然而然会被推迟；
- 即便当次任务在 1s 内执行完毕，`TimeUnit.MILLISECONDS.sleep` 也无法准确的让线程休眠指定毫秒，因为该方法内部还是调用的 `Thread.sleep`。

4.1.1 过期时间超过 5s 的任务

对于第一部分过期时间超过 5s 的任务，会根据任务配置的**调度过期策略**来选择要不要触发：

□ Java

JobScheduleHelper

```
1  if (nowTime > jobInfo.getTriggerNextTime() + PRE_READ_MS) {  
2      // 1、获取任务过期策略  
3      MisfireStrategyEnum misfireStrategyEnum = MisfireStrategyEnum.match(  
4      if (MisfireStrategyEnum.FIRE_ONCE_NOW == misfireStrategyEnum) {  
5          // 如果是 FIRE_ONCE_NOW，立即触发一次  
6          JobTriggerPoolHelper.trigger(jobInfo.getId(), TriggerTypeEnum.MI  
7      }  
8      // 2、更新该任务的下次触发时间  
9      refreshNextValidTime(jobInfo, new Date());  
10 }
```

调度过期策略*

忽略



任务超时时间*

忽略

立即执行一次

这两个策略就是字面意思：

- 直接忽略这个过期的任务；
- 立马触发一次这个过期任务。

4.1.2 过期时间小于 5s 的任务

对于第二部分过期时间小于 5s 的任务，会立马触发一次。如果判断下一次触发时间就在 5s 内，就将这个任务放到一个时间轮里，等待下一次触发执行：

□ Java

JobScheduleHelper

```
1  else if (nowTime > jobInfo.getTriggerNextTime()) {  
2      // 1. 立马触发一次任务，并更新下次触发时间
```

```

3      JobTriggerPoolHelper.trigger(jobInfo.getId(), TriggerTypeEnum.CRON,
4      refreshNextValidTime(jobInfo, new Date()));
5      // 2. 如果下次触发时间在未来五秒内，则将任务放到时间轮
6      if (jobInfo.getTriggerStatus()==1 && nowTime + PRE_READ_MS > jobInfo
7          // 1、make ring second
8          int ringSecond = (int)((jobInfo.getTriggerNextTime()/1000)%60);
9          // 2、push time ring
10         pushTimeRing(ringSecond, jobInfo.getId());
11         // 3、fresh next
12         refreshNextValidTime(jobInfo, new Date(jobInfo.getTriggerNextTime
13     }
14 }

```

4.1.3 未到触发时间的任务

对于第三部分任务，由于还没到触发时间，所以直接放到时间轮里等待处理。

到此，一次调度的计算就完成了。

4.2 时间轮的原理与实现

时间轮 (Time Ring) 是一种用于处理时间相关事件的数据结构，它会将时间划分为一系列的时间槽 (slots)，每个时间槽代表一段时间间隔，例如毫秒或秒。时间轮上有多个槽，它们形成一个环形结构，类似于钟表的刻度。

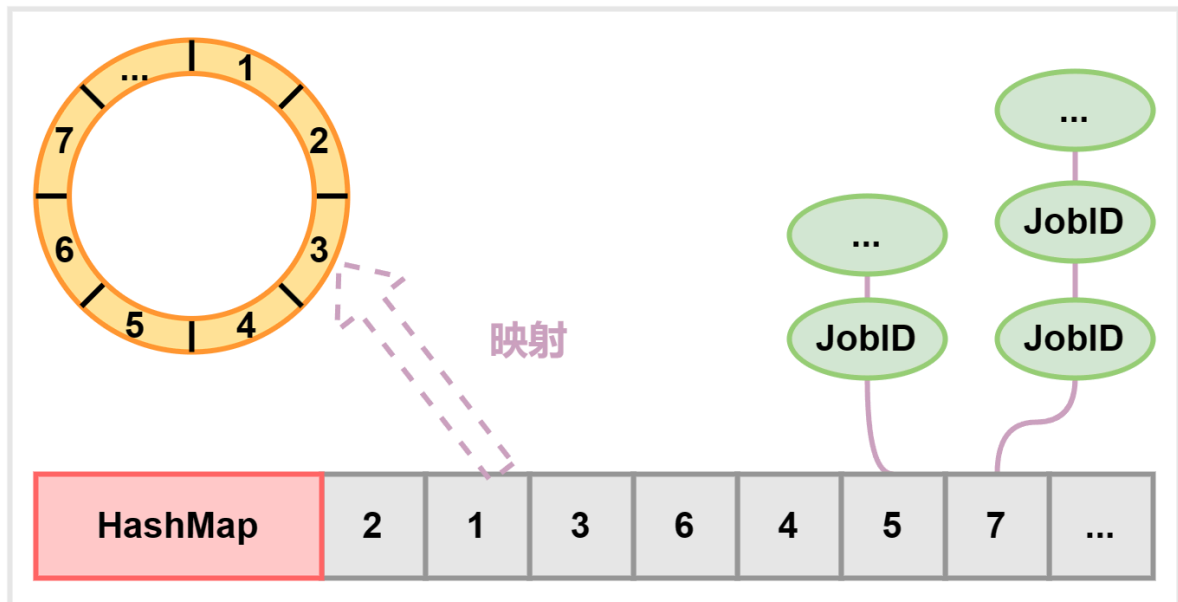
在具体的实现中，时间轮是一种线程安全的 HashMap，其中键 (Key) 表示时间刻度，而值 (Value) 则表示待执行的任务 ID 列表：

□ Java

JobScheduleHelper

```
1 private volatile static Map<Integer, List<Integer>> ringData = new Concu
```

示意图如下：



当调度线程将待执行任务放置到其下次触发时间所在的刻度上时，时间轮处理线程 `ringThread` 即可开始消费这些任务：

□ Java

JobScheduleHelper

```
1 while(...) {
2     // 执行频率控制...
3     try {
4         // 获取触发刻度内的任务
5         List<Integer> ringItemData = new ArrayList<>();
6         // 获取当前在分钟内的秒数, [0 - 59]
7         int nowSecond = Calendar.getInstance().get(Calendar.SECOND); //
8         for (int i = 0; i < 2; i++) {
9             List<Integer> tmpData = ringData.remove( (nowSecond+60-i)%60);
10            if (tmpData != null) {
11                ringItemData.addAll(tmpData);
12            }
13        }
14
15        // 触发到期任务的执行
16        logger.debug(">>>>>>>>> xxl-job, time-ring beat : " + nowSecond);
17        if (ringItemData.size() > 0) {
18            // do trigger
19            for (int jobId: ringItemData) {
20                // do trigger
21                JobTriggerPoolHelper.trigger(jobId, TriggerTypeEnum.CRON, 1, true, 1, false, jobId);
22            }
23            // clear
24            ringItemData.clear();
25        }
26    } catch ...
27 }
```

该时间轮一共划分了 **60** 个刻度，分别对应一分钟内的 60 秒。每次处理时，都会将**当前秒**跟**前一秒**这两个刻度的任务取下来处理。之所以要往前取一个刻度，是为了避免上次处理耗时超过了 1s，导致任务被遗漏。

□ 注意

与调度线程一样，时间轮处理线程也是尽可能**每秒**执行一次：

□ Java

JobScheduleHelper

```
1 while(...) {
2     // align second
3     try {
4         TimeUnit.MILLISECONDS.sleep(1000 - System.currentTimeMillis())
5     } catch (InterruptedException e) {
6         if (!ringThreadToStop) {
7             logger.error(e.getMessage(), e);
8         }
9     }
10
11     // 处理任务...
12 }
```

4.3 任务触发过程

无论是立即触发任务，还是交由时间轮去触发，最终都是由 `JobTriggerPoolHelper` 处理的。`JobTriggerPoolHelper` 是任务的异步触发器，它内部维护了一快一慢两个线程池：

□ Java

JobTriggerPoolHelper

```
1 private ThreadPoolExecutor fastTriggerPool = null;
2 private ThreadPoolExecutor slowTriggerPool = null;
```

`JobTriggerPoolHelper` 默认情况下会将任务交给 `fastTriggerPool` 处理，同时记录任务的**慢触发次数**（**触发时间超过 500ms**）：

这个**耗时**指的是执行器收到触发命令，并返回 HTTP 结果的耗时。因为执行器也是异步的，所以这个时间不包括任务的实际运行时间。

□ Java

JobTriggerPoolHelper

```
1 /*
2  com.xx1.job.admin.core.thread.JobTriggerPoolHelper#addTrigger
3  --> java.util.concurrent.ThreadPoolExecutor#execute
4  --> java.lang.Runnable#run
5  */
6 long cost = System.currentTimeMillis() - start;
7 if (cost > 500) { // ob-timeout threshold 500ms
8     AtomicInteger timeoutCount = jobTimeoutCountMap.putIfAbsent(jobId, n
```



```

9      if (timeoutCount != null) {
10         timeoutCount.incrementAndGet();
11     }
12 }

```

如果该任务一分钟内慢触发次数超过 10 次，就将这次触发任务交给 `slowTriggerPool` 处理：

□ Java

JobTriggerPoolHelper

```

1 // com.xx1.job.admin.core.thread.JobTriggerPoolHelper#addTrigger
2 ThreadPoolExecutor triggerPool_ = fastTriggerPool;
3 AtomicInteger jobTimeoutCount = jobTimeoutCountMap.get(jobId);
4 if (jobTimeoutCount!=null && jobTimeoutCount.get() > 10) { // job-timeou
5     triggerPool_ = slowTriggerPool;
6 }

```

快慢线程优化机制通过隔离操作，有效避免了慢任务阻塞其他任务的触发。

4.4 执行器的选择

`JobTriggerPoolHelper` 将待触发的任务提交给快慢线程池后，会由 `XXLJobTrigger` 完成任务的触发过程。由于执行器可能由多个实例，因此这里会通过路由策略选择执行器去执行：

路由策略*

第一个

调度过期策略*

任务超时时间*

第一个

最后一个

轮询

随机

一致性HASH

最不经常使用

最近最久未使用

故障转移

忙碌转移

分片广播

欢迎关注我的公众号，第一时间获取文章更新：



微信搜一搜

Geek编程社

相关内容

- 负载均衡策略与算法详解
- 分布式一致性原理详解
- CAP 模型与 ACID、BASE 理论
- Spring 配置 CORS 跨域资源共享的原理与实战
- Spring Security 配置 CSRF 防御的原理与实战

更新于 2024-03-12

CC BY-NC 4.0

□ Java